

17/06 2023 21/7/23

```
public static boolean isIn (Queue<Integer> q, int x) {  
    boolean bool = false;
```

```
    Queue<Integer> q1 = new CopyOnWriteArrayQueue<Integer>(q);
```

```
    while (!q1.isEmpty()) {
```

```
        if (q1.remove() == x)
```

```
            bool = true;
```

```
    }  
    return bool;
```

```
public static boolean twoSum (Queue<Integer> q, int x) {
```

```
    boolean bool = false;
```

```
    while (!q.isEmpty()) {
```

```
        if (isIn(q, x - q.remove()))
```

```
            bool = true;
```

```
    }  
    return bool;
```


27/02/2023 2:10 PM

(1c)

```
Public void insertNum(int num){
    Node<NumCount> p = this.L1;
    Node<NumCount> L1 = null;
    NumCount a;

    if (p == null || p.getValue().getNum() > num) {
        a = new NumCount(num, 1);
        L1 = new Node<NumCount>(a, L1);
    }
    else if (p.getValue().getNum() == num) {
        a = p.getValue();
        a.setCount(a.getCount() + 1);
    }
    else {
        while (p != null && p.getValue().getNum() < num) {
            L1 = p;
            p = p.getNext();
        }
        if (p != null && p.getNum() == num) {
            a = p.getValue();
            a.setCount(a.getCount() + 1);
        }
        else if (L1 != null && L1.getNum() < num) {
            a = L1.getValue();
            a.setCount(a.getCount() + 1);
        }
        else {
            a = new new NumCount(num, 1);
            L1.setNext(new Node<NumCount>(a, p));
        }
    }
}
```

3 3

קצת יותר

2. כ
מחלקת `Node` — מחלקת `Node` מכילה את מספר האיבר, סוגיית האיבר וקצת יותר.

Public int ValueN(int n) {

(2)

Node<NumCount> p=this.lst;

NumCount a;

while(p != null) {

a = p.getValue();

n = a.getCount();

if(n <= 0)

return a.getNum();

p = p.getNext();

3
return -1;

Public static boolean addNodes (BinNode<Integer> t) {

if (t == null)

return False;

int num = t.getValue();

if (isPrime(num))

return False;

int k = 2;

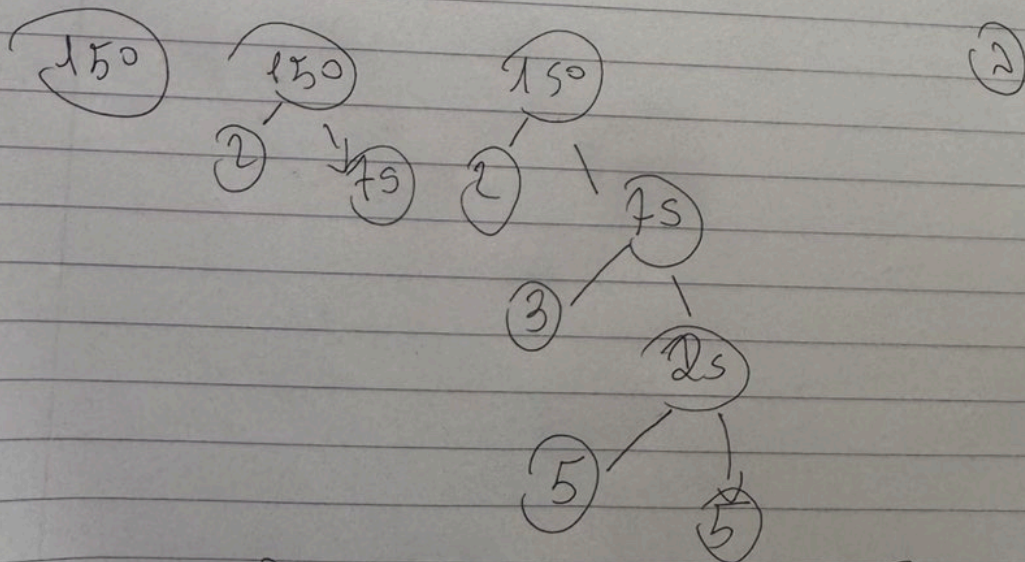
while (num % k != 0) {

k++;

f. setLeft (new BinNode<Integer> (k));

f. setRight (new BinNode<Integer> (num/k));

return True;



הערה: מה שכתבתי זה רק דוגמה
 למעשה צריך להוסיף עוד
 קוד כדי להבטיח שהעץ
 יהיה מאוזן.